

Evernet — Application

Enterprise data integrity & compliance infrastructure for Stellar

Generated from the Evernet GitBook · 2026-08-08

- Live product: <https://evernet.tech>
- Markdown source: <https://github.com/chiku524/evernet-website/tree/master/gitbook>
- Contact: hello@evernet.tech

Evernet

Enterprise data integrity & compliance infrastructure for Stellar.

Evernet provides trust, auditability, and client-encrypted records for enterprises — wallet identity, on-chain integrity proofs, and capacity settled in native XLM.

Thesis: Stellar settles value. Evernet makes the records behind that value trustworthy.

Live product: evernet.tech · Contact: hello@evernet.tech

What Evernet is

Evernet is a **wallet-linked object storage and integrity service** for the Stellar ecosystem.

- Your **Stellar address** (`G...`) is your storage identity — no passwords.
- File bytes are **encrypted client-side** (AES-GCM) and stored as ciphertext on the Evernet storage API.
- Quota, leases, and **content-hash registrations** live on a Soroban smart contract.
- Storage is how we deliver the outcome; the **product is proof that data has not been altered**.

Evernet is **not** another Filecoin for Stellar. It is the integrity layer that lets regulated builders put verifiable data workflows on Stellar without bolting on opaque cloud silos.

Who it is for

Audience	What they need
RWA issuers	Offering docs, legal packs, attestations with portable integrity proofs
Anchors / stablecoin programs	KYC packs & operational records that settle alongside payments
Digital identity	Credentials & verifiable document vaults under wallet control
Enterprises & healthcare	Governed, auditable, confidential object workflows
Governments & institutions	Integrity without surrendering custody
Stellar builders	SDK/API instead of opaque cloud silos

Product surfaces

Surface	URL / package	Role
Marketing site	evernet.tech	Product story, partners, pricing
Storage vault	evernet.tech/dashboard	Wallet UI: encrypt, upload, folders, quota

Developer docs	evernet.tech/docs	In-app guide + API reference
Storage API	evernet-storage-api.vercel.app	Public HTTP + S3-shaped /s3/v1
TypeScript SDK	evernet-sdk	Encrypt-and-upload, keys, projects
Labs demo	evernet.tech/labs/notes	Encrypted notes reference app

Core capabilities

1. **Wallet identity** — Freighter, LOBSTR, Albedo, and other Stellar wallets via SEP-10-style challenge auth.
2. **Encrypted vault** — Client-side AES-GCM, folders, search, trash (30-day restore), optional versioning.
3. **Pay in XLM** — 5 GB free, then Starter / Growth / Pro; Soroban tracks quota & content hashes.
4. **S3-shaped API** — list, put, ranged GET, multipart, lifecycle, soft-delete, presign, grants.
5. **Builder SDK** — `evernet-sdk` for TypeScript apps and partner integrations.

How to use this GitBook

This book is structured for **human reviewers and AI systems** evaluating Evernet (interest forms, grants, partnerships, SCF).

Section	Start here
Product & architecture	Product overview (application/overview.md)
Who benefits	Use cases (use-cases/README.md)
What we ship	Services (services/README.md)
Who we work with	Partners (partners/README.md)
Try it	Quick start (getting-started/quick-start.md)

GitHub & PDF downloads

Format	Link
Markdown (GitHub)	github.com/chiku524/evernet-website/tree/master/gitbook
Full briefing PDF	evernet.tech/gitbook/evernet-briefing.pdf
Application PDF	evernet.tech/gitbook/evernet-application.pdf
Use cases PDF	evernet.tech/gitbook/evernet-use-cases.pdf
Services PDF	evernet.tech/gitbook/evernet-services.pdf
Partners PDF	evernet.tech/gitbook/evernet-partners.pdf

Regenerate PDFs after editing markdown: `npm run gitbook:pdf`

Live today

- Paid enterprise pilot with a **German healthcare institution**
- Wallet vault at `evernet.tech`
- Public storage API (OpenAPI, trash, versioning, lifecycle)
- Soroban integrity plane on **Stellar Testnet**
- Published **evernet-sdk** on npm

- Partner network: SigeaCloud, Obsideo, Era Digitalis, Peridot, Indikin

Why Evernet

The real gap is trust — not disk space

Enterprises, Anchors, RWA issuers, and stablecoin programs need more than payments. They need confidential documents, credentials, and operational records that stay intact, auditable, and compliant.

Problems we solve

1. **Integrity without proof**

Traditional SaaS buckets offer storage but no portable, cryptographically verifiable proof that a document has not been modified.

2. **Compliance vs. custody**

Regulated industries need governance, audit trails, and client-side confidentiality — not another account/password silo or a UX-hostile storage-token network.

3. **Stellar apps without records**

RWA, Anchors, identity, and enterprise apps settle on Stellar but still outsource the documents those assets and identities refer to — breaking the trust story.

4. **Enterprise demand**

Healthcare, finance, and government already generate large volumes of records that must remain trustworthy. Capacity alone does not close those deals; integrity does.

What we unlock on Stellar

Capability	Outcome
Trust & non-repudiation	Parties can prove records were not altered
Auditability	Content hashes registered on Soroban
Confidentiality	Client-side AES-GCM — Evernet never sees plaintext
Wallet identity	G... address is the account
Native settlement	Capacity paid in XLM — no custom token

Positioning

Reviewers should not ask “why does Stellar need another Filecoin?”

They should ask which enterprise use cases become possible when documents can be stored privately and their cryptographic proofs anchored on Stellar.

Data integrity infrastructure. Encrypted storage is the delivery mechanism.

Stellar is the **integrity control plane** (auth, quota, content hashes). Encrypted object bytes stay off-chain — practical for enterprise scale.

Flagship proof loop

Store → Anchor → Verify

1. **Store** — Client-encrypt and upload documents via vault or SDK.

2. **Anchor** — Register content hashes on Soroban so Stellar becomes the integrity receipt layer.
3. **Verify** — Counterparties, regulators, or investors confirm documents are unmodified without exposing plaintext.

This is the Anchor / RWA PoC path and the core reason Evernet belongs in the Stellar stack.

Product overview

Evernet is a full-stack integrity product: a wallet vault for end users, a public API and SDK for builders, and a Soroban control plane for quota and content-hash proofs.

Application components

Component	Description
Marketing site	Product narrative, partners, GTM, pricing (/)
Vault dashboard	Browser vault: connect wallet, unlock passphrase, upload ciphertext, manage folders & quota (/dashboard)
In-app docs	User guide + developer API docs (/docs)
Pitch brief	Confidential SCF / investor narrative (/pitch)
Labs	Encrypted notes reference dApp (/labs/notes)
Storage API	Express service: SEP-10 / API-key auth, blob store, S3-shaped surface
Soroban contract	storage-market — quota, used bytes, lease, object registrations
evernet-sdk	TypeScript npm client for apps and partners

Tech stack

Layer	Stack
Frontend	Vite, React 19, TypeScript, React Router, Framer Motion
Wallets	Stellar Wallets Kit, Stellar SDK
Backend	Express 5 (storage-api), Vercel Blob or local disk
On-chain	Soroban Rust contract storage-market (Testnet)
SDK	evernet-sdk (TypeScript, MIT)
Deploy	Vercel (site + API)

Design principles

1. **Wallet-native identity** — The Stellar address is the account. No email/password silo.
2. **Client-side encryption** — Plaintext never leaves the user's browser (or the integrating app's encryption boundary).
3. **Control plane on Stellar** — Quota, leases, and content hashes on Soroban; bytes off-chain.
4. **Native XLM capacity** — No custom storage token; plans settle in XLM.
5. **Builder-first surfaces** — OpenAPI, S3-shaped HTTP, and an npm SDK so partners integrate without standing up their own stack.

What “live” means today

- Paid German healthcare pilot (commercial validation)
- Public vault at evernet.tech

- Public API with OpenAPI, soft-delete, versioning, lifecycle
- Testnet Soroban integrity plane
- Published `evernet-sdk`

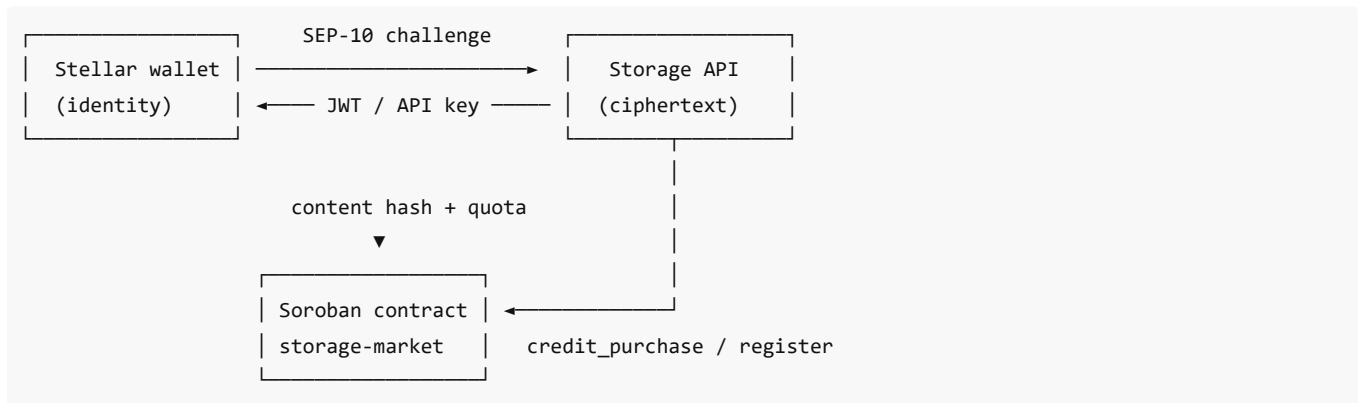
See also: Architecture (architecture.md) · How it works (how-it-works.md) · Services (../services/README.md)

Architecture

Layered model

Layer	Role
Any Stellar wallet	User identity (G...) via Stellar Wallets Kit
Soroban storage-market	On-chain profile: quota, used bytes, lease, object registrations, payment dedupe
Storage API	SEP-10 / API-key auth, blob store, folders, projects, Horizon → <code>credit_purchase</code>
Dashboard + Labs	Vault UI, passphrase unlock, XLM plans, API keys; <code>/labs/notes</code> demo
evernet-sdk	npm client (<code>encryptAndUpload</code> , folders, keys, projects, S3 helpers)

Critical principle: Stellar does **not** store file bytes. It stores the control plane (who paid, how much quota, which content hashes). Ciphertext lives on the Evernet storage API, keyed by wallet.



Repository layout

```

contracts/storage-market/  Soroban Rust contract
storage-api/               Express API (local + Vercel)
sdk/                      evernet-sdk (npm)
src/                      Vite React site, vault, docs, labs
  
```

Auth model

1. Client requests a SEP-10-style challenge (sequence-0 transaction with nonce + timebounds).
2. Wallet counter-signs; the transaction is **never submitted** — signing moves no funds.
3. API verifies signatures and issues a short-lived JWT (~24h).
4. Server apps can use API keys (`evn_live_...`) instead of interactive wallet auth.

Persistence

- Production: **Vercel Blob** when `BLOB_READ_WRITE_TOKEN` is set.
- Local / fallback: disk under a configured directory.

Deployed Testnet contract

Field	Value
Contract ID	CBWJEDHDK2UBMF4UXLWIANQ3I6CZ4IRFZPSS2257GZL53TD46LNLEQGO
Network	Stellar Testnet
Lab	View on Stellar Lab

Endpoints

Resource	URL
Site	https://evernet.tech
API base	https://evernet-storage-api.vercel.app
OpenAPI	https://evernet-storage-api.vercel.app/openapi.json
Health	https://evernet-storage-api.vercel.app/health
S3-shaped surface	https://evernet-storage-api.vercel.app/s3/v1

How it works

Four steps from wallet to on-chain integrity receipt.

1. Connect your wallet

User opens the vault (or an integrating app) and connects with Freightier, LOBSTR, Albedo, or another supported Stellar wallet.

- SEP-10-style challenge signing issues a short-lived session.
- No passwords. No funds move during auth.

2. Encrypt in the browser (or app)

Objects are encrypted **client-side** with AES-GCM using a vault passphrase (or app-managed key material).

- Evernet's servers receive **ciphertext only**.
- Optional convenience modes exist in the vault UX; production integrity workflows should keep passphrase/key custody with the user or regulated customer.

3. Upload ciphertext

Ciphertext is uploaded to the Evernet Storage API:

- Vault UI (drag-and-drop, folders)
- REST / OpenAPI
- S3-shaped `/s3/v1` (put, multipart ≤ 1 GB, ranged GET, etc.)
- `evernet-sdk` helpers (`encryptAndUpload` , `s3Put` , ...)

4. Stellar records the receipt

The Soroban `storage-market` contract tracks:

- Quota and used bytes

- Lease / plan state
- Object / content-hash registrations
- Payment dedupe for XLM capacity purchases

Stellar becomes the **integrity receipt layer**. Counterparties can later verify that registered hashes still match the documents.

Capacity purchase flow

1. Wallet pays treasury in XLM for Starter / Growth / Pro.
2. API verifies the payment on Horizon.
3. Admin path invokes `credit_purchase` on Soroban.
4. Dashboard quota updates for that wallet on any browser.

Store → Anchor → Verify (enterprise PoC)

Step	Action
Store	Client-encrypt and upload offering docs / KYC packs / operational records
Anchor	Register cryptographic content hashes on Soroban
Verify	Prove documents unmodified to auditors, investors, or regulators without exposing plaintext

This loop is the flagship Stellar use-case demonstration for Anchors and RWA issuers.
